

OS-Homework6

计算机 2303 尤颢辰

2026 年 5 月 13 日

1 Question 1:

自旋锁不适合单处理器系统，因为等待锁的线程会一直占用唯一的 CPU，导致持有锁的线程无法运行并释放锁，从而造成长时间等待。而在多处理器系统中，持有锁的线程可以在另一个处理器上继续执行并很快释放锁，因此等待线程短时间自旋可以避免睡眠、唤醒和上下文切换的开销。

2 Question 2:

如果 wait 和 signal 原语不是原子操作，互斥性可能会被破坏。我们假设 wait 操作形如：

```
1 wait ()
2 {
3     while(mutex);
4     mutex--;
5 }
6 进入临界区 ...
```

我们假设再执行完 while(mutex) 语句之后，发生了中断，转为执行另一个进程。那么这个进程也会通过 while(mutex); 的检查。这样就导致两个进程可能同时访问临界区资源，就会破坏互斥性。

3 Question 3:

定义三个信号量，一个共享变量：

int waiting = 0 用于记录等待中的顾客数；

semaphore mutex = 1 用于保证对 waiting 的互斥访问；

semaphore customers = 0 等待理发的顾客数量；

semaphore barbers = 0 有多少个理发师可以叫号；

理发师进程：

```
1 void barber()
2 {
3     while (true)
4     {
5         wait(customers);
```

```

6
7     wait(mutex);
8     waiting--;          // 叫一个等待的顾客进理发室
9     signal(barbers);    // 通知一个顾客可以理发
10    signal(mutex);
11
12    cut_hair();
13 }
14 }

```

顾客进程:

```

1 void customer(
2 {
3     wait(mutex);
4     if (waiting < N)
5     {
6         waiting++;      // 顾客等待
7         signal(customers); // 通知理发师
8         signal(mutex);
9         wait(barbers);   // 等待理发师叫自己
10        get_haircut();
11    }
12    else
13    {
14        signal(mutex);   // 离开
15        leave_shop();
16    }
17 }

```

4 Question 4:

1. 在 V 操作之中不能从等待队列尾部唤醒进程，因为这可能导致部分进程长期得不到 CPU 资源，产生饥饿现象。同时，还应该保证 PV 是原子操作。
2. 在这个情况下，我们需要保证对于共享变量的访问是互斥的。

定义信号量 semaphore mutex = 1，用于保证对共享变量的互斥访问。

每个进程的代码框架如下：

```

1 void threads()
2 {
3     P(mutex);
4     访问X... ..;
5     V(mutex);
6 }
7

```

5 Question 5:

我们先分析一下问题在哪。假设有源源不断的读进程要读，那么 readcount 就会一直大于 0，这样写进程就会被一直阻塞。

我们定义两个共享变量和四个信号量：

int readcount = 0 记录有几个读进程

int writecount = 0 记录有几个写进程；

semaphore Rmutex = 1 实现对 readcount 的互斥访问；

semaphore Wmutex = 1 实现对 writecount 的互斥访问；

semaphore ReadBlock = 1 实现写进程优先；

semaphore WriterResource = 1 实现写进程互斥；

代码框架如下：

写进程：

```
1 void writer()
2 {
3     P(Wmutex)
4     if(writecount == 0) P(ReadBlock);
5     writecount++;
6     V(Wmutex)
7     P(WriterResource)
8     写 ... ..
9     V(WriterResource)
10    P(Wmutex)
11    writecount--;
12    if(writecount == 0) V(ReadBlock);
13    V(Wmutex)
14 }
```

读进程：

```
1 void reader()
2 {
3     P(ReadBlock)
4     P(Rmutex)
5     if(readcount == 0) P(WriterResource);
6     readcount++;
7     V(Rmutex)
8     V(ReadBlock)
9     读 ... ..
10    P(Rmutex)
11    readcount--;
12    if(readcount == 0) V(WriterResource);
13    V(Rmutex)
14 }
```

6 Question 6:

1. 一共需要设计 $N+1$ 个进程，其中 N 个学生进程，1 个教师进程。

2. `int incount = 0;` // 已进入考场的人数

`semaphore door = 1;` // 控制考场门口互斥

`semaphore mutex1 = 1;` // 保护 `incount`

`semaphore allin = 0;` // 是否所有学生都进入

`semaphore papers = 0;` // 学生交卷人数

`semaphore examstart = 0;` // 教师发卷后开始考试

学生进程：

```
1 void student()
2 {
3     P(door);
4     进入考场;
5     V(door);
6     P(mutex1);
7     incount++;
8     if (incount == N) V(all_in); // 通知教师开始考试
9     V(mutex1);
10    P(examstart)
11    考试;
12    交卷;
13    V(papers);
14    P(door);
15    离开考场;
16    V(door);
17 }
18
```

教师进程：

```
1 void teacher()
2 {
3     P(door);
4     进入考场;
5     V(door);
6     P(allin);
7     for (0:N) V(examstart);
8     for (0:N) P(papers);
9     封装试卷;
10    P(door);
11    离开考场;
12    V(door);
13 }
14
```